

프로그래밍 프로젝트 - 배부 자료 기준 10 문제 풀이

- 과목: 프로그래밍 수행평가
- 작성: 3 0 8 0 1 권순찬
- 작성 환경: Google Colab / Python

배부 자료에 제시된 문항 번호를 그대로 유지했습니다. 사진 자료에는 문제 3이 보이지 않고 문제 1 1 (선택 정렬)까지 제시되어 있으므로, 총 10 문항인 1, 2, 4, 5, 6, 7, 9, 10, 11 번을 풀이했습니다.

각 문항은 문제 요약 → 코드 → 실행 결과 순서로 구성했습니다.

In [1]:

```
# 실행 예시에서 input()을 자동으로 재현하기 위한 보조 함수입니다.
def make_input(values):
    values = iter(values)
    def fake_input(prompt=''):
        value = str(next(values))
        print(f"{prompt}{value}")
        return value
    return fake_input
```

문제 1. 좌표가 위치한 사분면 출력하기

x, y 좌표를 입력받아 if - elif - else 조건문으로 어느 사분면에 있는지 출력합니다.

배부 자료의 조건처럼 x= 0 또는 y= 0 인 경우는 제외합니다.

In [2]:

```
def quadrant_program(input_fn=input):
    x = int(input_fn("x좌표를 입력하세요: "))
    y = int(input_fn("y좌표를 입력하세요: "))

    if x > 0 and y > 0:
        print("제1사분면에 있습니다.")
    elif x < 0 and y > 0:
        print("제2사분면에 있습니다.")
    elif x < 0 and y < 0:
        print("제3사분면에 있습니다.")
    else:
        print("제4사분면에 있습니다.")

# 실행 예시
quadrant_program(make_input([-100, 200]))
```

x좌표를 입력하세요: -100
y좌표를 입력하세요: 200
제2사분면에 있습니다.

문제 2. 연도와 월을 입력받아 그 달의 일수 출력하기

윤년은 다음 기준으로 판단합니다.

- 4 0 0의 배수이면 윤년
- 4의 배수이면서 1 0 0의 배수가 아니면 윤년

In [3]:

```
def days_in_month_program(input_fn=input):
    year = int(input_fn("연도를 입력하세요: "))
    month = int(input_fn("월을 입력하세요: "))

    leap = (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0)
    days = [31, 29 if leap else 28, 31, 30, 31, 30,
            31, 31, 30, 31, 30, 31]

    if 1 <= month <= 12:
        print(f"{year}년 {month}월의 마지막 날은 {days[month-1]}일입니다.")
    else:
        print("월은 1부터 12 사이로 입력해야 합니다.")

# 배부 자료의 입력 예시: 2008년 10월
days_in_month_program(make_input([2008, 10]))
```

연도를 입력하세요: 2008
월을 입력하세요: 10
2008년 10월의 마지막 날은 31일입니다.

문제 4. 랜덤 모듈과 함수를 이용한 가위바위보 게임

사용자가 가위/바위/보를 입력하면 컴퓨터가 무작위로 하나를 선택하고, 비김/사용자 승리/사용자 패배를 출력

In [4]:

```
import random

def rsp_game(input_fn=input):
    choices = ["가위", "바위", "보"]
    user = input_fn("가위/바위/보 중 하나를 입력하세요: ")

    if user not in choices:
        print("가위, 바위, 보 중 하나만 입력하세요.")
        return

    computer = random.choice(choices)
    print("컴퓨터:", computer)

    if user == computer:
```

```

        print("비겼습니다.")
    elif (user == "가위" and computer == "보") or \
          (user == "바위" and computer == "가위") or \
          (user == "보" and computer == "바위"):
        print("사용자가 이겼습니다.")
    else:
        print("사용자가 졌습니다.")

# 실행 결과를 재현하기 위해 시드 고정
random.seed(7)
rsp_game(make_input(["바위"]))

```

가위/바위/보 중 하나를 입력하세요: 바위
 컴퓨터: 바위
 비겼습니다.

문제 5. 조건에 맞는 동물 이름 리스트 만들기

- 두 글자 이상의 동물 이름만 추가
- 이미 리스트에 있는 동물은 다시 추가하지 않음
- 아무것도 입력하지 않고 Enter를 누르면 종료
- 종료 후 오름차순으로 정렬하여 출력

In [5]:

```

def animal_list_program(input_fn=input):
    animals = []

    while True:
        animal = input_fn("동물을 입력하세요 (끝내려면 Enter): ").strip()

        if animal == "":
            break
        if len(animal) < 2:
            print("두 글자 이상의 동물이름을 입력하세요.")
        elif animal in animals:
            print(f"{animal}은(는) 이미 동물리스트에 들어있습니다.")
        else:
            animals.append(animal)

    animals.sort()
    print("최종 동물 리스트:", animals)

# 배부 자료 화면의 흐름을 재현한 실행 예시
animal_list_program(make_input(["호랑이", "뱀", "사자", "사자", ""]))

```

동물을 입력하세요 (끝내려면 Enter): 호랑이
 동물을 입력하세요 (끝내려면 Enter): 뱀
 두 글자 이상의 동물이름을 입력하세요.
 동물을 입력하세요 (끝내려면 Enter): 사자
 동물을 입력하세요 (끝내려면 Enter): 사자
 사자은(는) 이미 동물리스트에 들어있습니다.
 동물을 입력하세요 (끝내려면 Enter):
 최종 동물 리스트: ['사자', '호랑이']

문제 6. 10 ~ 99 사이의 두 자리 숫자 맞추기 게임

컴퓨터가 10 ~ 99 사이의 숫자를 하나 정합니다.

두 자리 숫자를 입력한 경우에만 기회를 소모하고, 정답을 맞힐 때까지 반복한 뒤 몇 번 만에 맞혔는지 출력함

In [6]:

```
def number_guess_game(input_fn=input, answer=None):
    if answer is None:
        answer = random.randint(10, 99)

    count = 0
    print("10과 99사이의 숫자 하나를 맞춰보세요")

    while True:
        value = input_fn(f"기회#{count+1}: ")

        if not value.isdigit() or not (10 <= int(value) <= 99):
            print(f"{value}는 두자리 숫자가 아닙니다")
            continue

        guess = int(value)
        count += 1

        if guess == answer:
            print("맞혔어요")
            print(f"{count}번만에 맞혔네요")
            break
        elif guess < answer:
            print("작네요. 좀 더 큰 수를 입력하세요")
        else:
            print("크네요. 좀 더 작은 수를 입력하세요")

# answer 인자는 실행 예시를 재현하기 위한 것으로, 생략하면 랜덤 숫자가 생성됩니다.
number_guess_game(make_input(["ab", "9a", "90", "94"]), answer=94)
```

10과 99사이의 숫자 하나를 맞춰보세요

기회#1: ab

ab는 두자리 숫자가 아닙니다

기회#1: 9a

9a는 두자리 숫자가 아닙니다

기회#1: 90

작네요. 좀 더 큰 수를 입력하세요

기회#2: 94

맞혔어요

2번만에 맞혔네요

문제 7. 좋아하는 이미지를 2차원 리스트로 만들어 출력하기

2차원 리스트의 0과 1을 이용해 간단한 웃는 얼굴 이미지를 만들고 출력합니다.

In [7]:

```
smile = [
    [0,0,1,1,1,1,1,0,0],
    [0,1,0,0,0,0,0,1,0],
    [1,0,1,0,0,0,1,0,1],
    [1,0,0,0,0,0,0,0,1],
    [1,0,1,0,0,0,1,0,1],
    [1,0,0,1,1,1,0,0,1],
    [0,1,0,0,0,0,0,1,0],
    [0,0,1,1,1,1,1,0,0]
]

for row in smile:
    print(" ".join("★" if cell else "☆" for cell in row))
```

```
☆ ☆ ★ ★ ★ ☆ ☆ ☆
☆ ★ ☆ ☆ ☆ ☆ ☆ ☆
★ ☆ ★ ☆ ☆ ☆ ☆ ★
★ ☆ ☆ ☆ ☆ ☆ ☆ ☆
★ ☆ ★ ☆ ☆ ☆ ☆ ☆
★ ☆ ☆ ★ ★ ☆ ☆ ☆
☆ ★ ☆ ☆ ☆ ☆ ☆ ☆
☆ ☆ ★ ★ ★ ★ ☆ ☆
```

문제 8. 랜덤 개인정보 생성 함수 만들기

랜덤으로 이름, 나이, e-mail, 전화번호, 성별을 생성하는 함수를 작성합니다.

배부 자료의 조건대로 각 함수는 `return` 을 사용합니다.

In [8]:

```
import csv
import string

last_names = ["김", "이", "박", "최", "정", "강", "조", "윤", "장", "임"]
first_names = ["민준", "서준", "도윤", "예준", "시우", "서연", "서윤", "지우", "하윤",
               "민서"]

def random_name():
    return random.choice(last_names) + random.choice(first_names)

def random_age():
    return random.randint(15, 60)

def random_mail(length=8):
    chars = string.ascii_lowercase + string.digits
    account = ''.join(random.choice(chars) for _ in range(length))
```

```

    return account + "@example.com"

def random_phone():
    return f"010-{random.randint(0,9999):04d}-{random.randint(0,9999):04d}"

def random_gender():
    return random.choice(["male", "female"])

def random_profile():
    return {
        "name": random_name(),
        "age": random_age(),
        "e-mail": random_mail(),
        "phone": random_phone(),
        "gender": random_gender()
    }

random.seed(30801)
profiles = [random_profile() for _ in range(10)]

for p in profiles:
    print(p)

```

```

{'name': '정예준', 'age': 46, 'e-mail': 'ojn8tanq@example.com', 'phone': '010-0917-5831', 'gender': 'male'}
{'name': '이민서', 'age': 48, 'e-mail': 'qr6bt5uf@example.com', 'phone': '010-9039-3911', 'gender': 'male'}
{'name': '김서준', 'age': 30, 'e-mail': 'nt5jl028@example.com', 'phone': '010-0144-4915', 'gender': 'female'}
{'name': '조도윤', 'age': 56, 'e-mail': 'llnvc2nf@example.com', 'phone': '010-9692-1317', 'gender': 'female'}
{'name': '강도윤', 'age': 34, 'e-mail': 'h1b7zps1@example.com', 'phone': '010-7697-7182', 'gender': 'male'}
{'name': '이서윤', 'age': 59, 'e-mail': 'xxij1xal@example.com', 'phone': '010-8163-9703', 'gender': 'male'}
{'name': '정지우', 'age': 33, 'e-mail': '2v1xrfn6@example.com', 'phone': '010-1883-9296', 'gender': 'male'}
{'name': '김민서', 'age': 30, 'e-mail': 'r0s7ntl6@example.com', 'phone': '010-1754-2344', 'gender': 'female'}
{'name': '장서연', 'age': 56, 'e-mail': '1cpfavaw@example.com', 'phone': '010-5959-8396', 'gender': 'female'}
{'name': '장지우', 'age': 45, 'e-mail': '9xosezf2@example.com', 'phone': '010-7829-1356', 'gender': 'female'}

```

문제 9. 생성된 결과를 CSV 파일로 저장하기

배부 자료에는 “1 번 문제의 생성된 결과”라고 적혀 있지만, 같은 쪽의 예시 CSV가

`name, age, e-mail, phone, gender` 열로 구성되어 있으므로 문제 8 에서 생성한 개인정보 결과로 저장했습니다.

파일명은 자신의 학번을 사용해 `30801.csv` 로 지정했습니다.

In [9]:

```

filename = "30801.csv"

with open(filename, "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "age", "e-mail", "phone",
"gender"])
    writer.writeheader()
    writer.writerows(profiles)

print(f"{filename} 파일 저장 완료")
print("저장 내용 확인:")
with open(filename, "r", encoding="utf-8-sig") as f:
    for i, line in enumerate(f):
        print(line.rstrip())
        if i >= 5:
            print("...")
            break

```

30801.csv 파일 저장 완료

저장 내용 확인:

name,age,e-mail,phone,gender

정예준,46,ojn8tanq@example.com,010-0917-5831,male

이민서,48,q6bt5uf@example.com,010-9039-3911,male

김서준,30,nt5jl028@example.com,010-0144-4915,female

조도윤,56,llnvc2nf@example.com,010-9692-1317,female

강도윤,34,h1b7zps1@example.com,010-7697-7182,male

...

문제 10. 버블 정렬

주어진 리스트 [12, 5, 8, 21, 1, 15, 7, 3, 10, 18] 을 버블 정렬로 오름차순 정렬합니다.

배부 자료의 코드 틀에 맞춰 기본 버블 정렬과 개선된 버블 정렬을 모두 구현했습니다.

In [10]:

```

array = [12, 5, 8, 21, 1, 15, 7, 3, 10, 18]

def bubble_sort(data):
    a = data.copy()
    n = len(a)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
        print(f"단계{i+1}: {a}")
    return a

def improved_bubble_sort(data):
    a = data.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
    return a

```

```

        print(f"단계{i+1}: {a}")
        if not swapped:
            break
    return a

print("원본:", array)
print()
print("[버블 정렬]")
result1 = bubble_sort(array)
print("정렬 결과:", result1)
print()
print("*" * 40)
print("[개선된 버블 정렬]")
result2 = improved_bubble_sort(array)
print("정렬 결과:", result2)

```

원본: [12, 5, 8, 21, 1, 15, 7, 3, 10, 18]

[버블 정렬]

단계1: [5, 8, 12, 1, 15, 7, 3, 10, 18, 21]
 단계2: [5, 8, 1, 12, 7, 3, 10, 15, 18, 21]
 단계3: [5, 1, 8, 7, 3, 10, 12, 15, 18, 21]
 단계4: [1, 5, 7, 3, 8, 10, 12, 15, 18, 21]
 단계5: [1, 5, 3, 7, 8, 10, 12, 15, 18, 21]
 단계6: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 단계7: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 단계8: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 단계9: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 정렬 결과: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]

[개선된 버블 정렬]

단계1: [5, 8, 12, 1, 15, 7, 3, 10, 18, 21]
 단계2: [5, 8, 1, 12, 7, 3, 10, 15, 18, 21]
 단계3: [5, 1, 8, 7, 3, 10, 12, 15, 18, 21]
 단계4: [1, 5, 7, 3, 8, 10, 12, 15, 18, 21]
 단계5: [1, 5, 3, 7, 8, 10, 12, 15, 18, 21]
 단계6: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 단계7: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 정렬 결과: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]

문제 1 1. 선택 정렬

주어진 리스트 [12, 5, 8, 21, 1, 15, 7, 3, 10, 18] 에서 매 단계 가장 작은 값을 찾아 기준 위치 오름차순으로 정렬합니다.

In [11]:

```

array = [12, 5, 8, 21, 1, 15, 7, 3, 10, 18]

def selection_sort(data):
    a = data.copy()
    n = len(a)

    for i in range(n - 1):
        min_idx = i

```

```

        for j in range(i + 1, n):
            if a[j] < a[min_idx]:
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i]
        print(f"단계{i+1}: {a}")

    return a

print("원본:", array)
result = selection_sort(array)
print("정렬 결과:", result)

```

원본: [12, 5, 8, 21, 1, 15, 7, 3, 10, 18]
 단계1: [1, 5, 8, 21, 12, 15, 7, 3, 10, 18]
 단계2: [1, 3, 8, 21, 12, 15, 7, 5, 10, 18]
 단계3: [1, 3, 5, 21, 12, 15, 7, 8, 10, 18]
 단계4: [1, 3, 5, 7, 12, 15, 21, 8, 10, 18]
 단계5: [1, 3, 5, 7, 8, 15, 21, 12, 10, 18]
 단계6: [1, 3, 5, 7, 8, 10, 21, 12, 15, 18]
 단계7: [1, 3, 5, 7, 8, 10, 12, 21, 15, 18]
 단계8: [1, 3, 5, 7, 8, 10, 12, 15, 21, 18]
 단계9: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]
 정렬 결과: [1, 3, 5, 7, 8, 10, 12, 15, 18, 21]